PROJECT REQUIREMENTS REPORT

White-Label Quotation & MOU Creation and Digital Signing Platform

Developer hand off specification • Version 1.0 • 27 October 2026

End Date - 31 October 2026

Document purpose	Product direction
Define the product scope, user model, functional requirements, technical stack, UX expectations, security requirements, and acceptance criteria for implementation.	A multi-tenant SaaS platform that lets organizations create branded quotations and MOUs, share secure links, and collect signatures without requiring external recipients to create accounts.

Important product principle

Organization-first and white-label by design. The organization owns its workspace configuration, branding, templates, members, and documents. External recipients are document-specific participants, not workspace users by default.

1. Executive Summary

Build a modern, secure, organization-based SaaS application for creating quotations and Memorandums of Understanding (MOUs), rendering them on company letterheads, sharing them through secure links, and collecting electronic signatures. Each organization must be able to present a branded experience using its logo, colors, company details, letterhead, and optional branded login/registration pages.

The platform must support multiple independent organizations (multi-tenancy), an internal platform administrator, organization administrators, and editors. External recipients must be able to open a document link, review the document, and sign without registering by default. An organization may optionally enable registration for people who need an account within that organization's branded workspace.

2. Goals and Non-Goals

2.1 Goals

• Make organization onboarding and brand setup straightforward.

• Generate polished, consistent quotations and MOUs from reusable templates.

• Support company letterheads and logos without content overlapping the design.

• Allow secure document sharing, recipient review, signing, and final document retrieval.

• Provide clear status tracking and an audit trail for each document.

• Keep every organization's users, settings, assets, documents, and audit records isolated. •

Deliver a responsive, premium UI that feels like a professional white-label product.

2.2 Non-goals for the initial release

• A full general-purpose word processor or unrestricted page-layout design tool.

• A complete accounting, invoicing, CRM, or payment collection system.

• Guaranteeing that every MOU is legally valid or replacing legal review.

• Claiming that a drawn/typewritten signature is equivalent to a certificate-based digital signature. •

Building every possible document type before the core quotation/MOU lifecycle works reliably.

3. Product Roles and Access Model

Do not confuse a document recipient with a registered organization account. The roles and permissions must be enforc on the server.

Actor	Purpose and permitted scope
Platform Admin (internal)	Operates the SaaS itself: manages organizations, platform configuration, subscription/usage controls if implemented, support and abuse review. Must not be treated as an organization member by default.
Organization Admin	Owns/manages one organization's workspace. Configures branding, letterheads, templates, organization settings, members, optional registration policy, and approval/signing workflow. Can view/manage organization documents according to policy.
Editor	An authenticated workspace member who can prepare and edit drafts and use approved templates. Permissions for issuing/sending documents should be configurable; editor access must not automatically grant organization settings or member-management privileges.
External Recipient	A person invited through a specific document link. Can view that document and perform permitted actions such as sign, decline, or request changes. No account is required by default, and the recipient must not gain workspace-wide access.
Optional Registered Organization User	Only exists if the organization enables registration. Account membership and access must be governed by the organization's chosen mode and permissions. Registration must not automatically expose all organization data or documents.

3.1 Optional organization registration settings

• Disabled / invitation-only:onlyinvitedworkspacememberscan sign in.

• Registration with admin approval: users may request access; an organization admin approves or rejects the request.

• Open registration: organization can explicitly enable automatic membership, with a defined default role and tightly scoped permissions.

• The setting must be off by default unless product policy explicitly chooses otherwise.

• Organization registration pages must be branded, but branding is not a security boundary. Tenant identity must be resolved and verified server-side.

4. Organization Onboarding and White-Label Setup

• Public SaaS landing page explaining the product, features, plans (if monetization is included), and organization sign-up.

• Organization sign-up form with organization name, business contact details, admin name, email, password/identity setup, and required consent.

• Create an organization workspace and assign the initial Organization Admin.

• Collect logo, brand colors, company legal/display name, address, contact information, website, tax/business identifiers where applicable, and authorized signatory details.

• Allow letterhead/template upload and preview. Validate file type, size, page dimensions, orientation, and rendering compatibility.

• Provide branded login and optional registration pages for each organization.

• Support organization-level branding in application screens, emails where practical, document previews, and generated documents.

• Custom domains may be a later phase; design the tenant/domain resolution model so this can be added safely.

5. Core Functional Requirements

5.1 Company assets and letterhead management

CORE REQUIREMENT — SAME COMPANY LETTERHEAD FOR BOTH DOCUMENT TYP

Boththe MOU makerandquotationmaker must use theexact sameactiveletterhead/templateconfiguredbythe organization. The user must not have to upload a separate letterhead for each document type. The organization can up or select its active letterhead centrally, and the same approved asset is used by both document workflows.

CONTENT EDITOR: Provide an editor for the document body. Users compose and format the quotation or MOU content in the editor; the company letterhead is applied as the document design/background or layout layer. The editor must not unintentionally overwrite, distort, or remove the organization's letterhead.

LIVE PREVIEW: Provide a preview before finalizing or sharing. The preview must show the actual content rendered on the selected letterhead, including page breaks, margins, tables, totals, clauses, page numbers, and signature blocks. Users must be able to return to editing and preview again.

SHARE AND DOWNLOAD: After preview and confirmation, users can generate the final PDF, download it, and create a secure shareable link for the recipient. The recipient sees the same issued version, can review it, and can complete the configured signing workflow. Preserve the issued version so subsequent edits do not silently change a shared or signed document.

MULTI-PAGE HANDLING: For a multi-page MOU or quotation, apply the configured letterhead consistently on every page where intended, respect header/footer safe areas, and prevent text from overlapping logos, letterhead text, footer content, tables, or signature blocks.

• Upload and manage logo, letterhead PDF/image, and reusable document assets.

• Preview assets before setting them as active.

• Support replacing assets while preserving historical versions needed to reproduce previously issued documents.

• Configure page size, margins, header/footer zones, logo placement, and signatory block placement where supported.

• Render multi-page documents consistently; avoid text colliding with letterhead headers, footers, logos, or page numbers.

• Store uploaded files privately. Use authorization-checked download URLs or short-lived signed URLs.

5.2 Quotationbuilder

• Create, save, edit, duplicate, preview, issue, revise, archive, and search quotations.

• Generate a unique quotation number using an organization-configurable numbering pattern.

• Include issue date, expiry/validity date, customer/recipient details, items/services, descriptions, quantities, units, rates, discounts, taxes, subtotal, total, currency, payment terms, delivery terms, notes, and attachments as appropriate.

• Support configurable tax fields; do not assume all organizations have the same tax or GST requirements.

• Provide reusable item/service descriptions and reusable quotation templates.

• Allow draft review and optional internal approval before sending.

• Preserve sent versions. Revisions should create a new version or revision record rather than silently altering the original.

5.3 MOU builder

• Create MOUs from structured templates and editable fields/clauses.

• Support parties' legal names and addresses, purpose, scope, responsibilities, deliverables, financial terms where applicable, effective date, duration, termination, confidentiality, other configurable clauses, annexures, and signatory blocks.

• Support two or more parties/signatories where the workflow requires it.

• Allow reusable clause libraries and organization-approved templates.

• Support draft, internal review, issued, awaiting signature, completed, declined, expired, revoked, and superseded states where applicable.

• Include a clear notice that templates are not legal advice and that legal effect and execution requirements depend on the document and applicable law.

5.4 Document preview and PDF generation

• Generate a faithful preview before issue.

• Generate downloadable PDFs using server-side rendering so the result is consistent across devices.

• Apply the selected organization letterhead and branding without breaking document layout.

• Support multi-page documents, tables that continue across pages, page numbers where configured, and signature blo that do not split awkwardly.

• Keep an exact issued-document snapshot/version for signing and evidence.

• Validate generated files and handle rendering failures with clear error messages and retryable processing.

5.5 Secure document sharing and recipient experience

• Generate a unique, cryptographically strong, unguessable link for each document recipient/workflow.

• Support link expiry, revocation, and controlled reissue.

• Allow the recipient to open the document without creating an account by default.

• Show the organization branding and the document's relevant context on the recipient page.

• Allow the recipient to review the complete document and then sign, decline, or request changes where enabled.

• For sensitive documents, support optional recipient verification (for example, email OTP) based on organization policy.

• Prevent link recipients from browsing other documents, searching the workspace, or accessing organization settings.

• Provide clear completion confirmation and access to the final document according to the organization's sharing policy.

SCOPE SIMPLIFICATION: For the initial version, recipients simply upload an image of their signature and place it in the signature area. Do not implement complex e-signature provider integrations or MFA. Preserve the issued document version and record basic signing history.

5.6 Electronicsigning

• Provide a simple, mobile-friendly signing experience. The recipient should be able to upload an image of their signature (for example, PNG or JPG) and position it in the designated signature area.

• Allow the recipient to upload a signature image and apply it to the document. Keep the signature image and the final signed PDF associated with the specific document version.

• Do not build third-party eSign provider integrations for the initial version. Keep the signing workflow simple and limited to signature-image upload and placing the signature on the document.

• Support the required signature fields for the document. Keep multi-party signing order out of scope unless specifically requested later.

• Record the signer-provided name (if collected), signing timestamp, document version, and basic workflow history. Avoid collecting unnecessary identity data.

• After completion, preserve the signed artifact and its evidence record; any later change must create a new version and must not silently replace the signed file.

• Describe this feature as adding a signature image to a document. Do not claim that an uploaded signature image is equivalent to a certificate-based digital signature or guarantees legal validity.

• Provide clear feedback for invalid uploads, failed saves, expired/revoked document links, and unsuccessful PDF generation.

5.7 Document dashboard and notifications

• Show counts and lists for drafts, awaiting internal approval, sent, awaiting signature, completed, declined, expired, revoked, and archived documents.

• Provide search and filters by document type, number, recipient, date, creator, status, and template where applicable.
• Show a document timeline with significant events and who performed them.

• Send email notifications for document sharing, signing completion, decline, revision request, and reminders where enabled.

• Use background jobs for PDF rendering, reminders, and email delivery; implement retries and avoid duplicate notifications.

• Provide configurable reminder frequency and stop reminders when a workflow is completed, declined, expired, or revoked.

6. User Interface and Experience Requirements

The UI must be premium, modern, responsive, and production-quality. Use a coherent design system rather than applying visual effects randomly. The requested aesthetic combines restrained glassmorphism and subtle neuromorphism with clean typography, smooth edges, generous spacing, and polished motion.

• Use rounded cards and controls, soft depth, subtle borders, tasteful translucency, and consistent spacing/radius tokens.

• Use glass effects selectively for navigation, overlays, or emphasis. Keep primary forms, document content, and data tables readable and accessible.

• Use subtle transitions and micro-interactions for navigation, modals, menus, previews, status changes, and loading states.

• Avoid clutter, excessive gradients, heavy shadows, excessive blur, noisy backgrounds, gratuitous animation, and inconsistent rounded corners.

• Do not sacrifice performance, contrast, accessibility, or document readability for visual effects.

• Design for mobile, tablet, laptop, and large desktop breakpoints.

• Include polished empty states, skeleton/loading states, error states, success confirmations, validation messages, and confirmation dialogs for destructive actions.

• Provide keyboard navigation, visible focus states, semantic controls, accessible labels, and reduced-motion support.

• Use consistent iconography, typography, button hierarchy, form layout, spacing, and status colors.

• The document preview and recipient signing page must prioritize legibility and trust over decorative effects.

7. Required Technology Stack

Layer	Required technology / approach	Implementation notes
Frontend	React + Next.js + TypeScript	Use Next.js routing and rendering features appropriately. Use strict TypeScript and reusable components.
Styling	Tailwind CSS	Build a shared design system with tokens for color, spacing, radius, shadows, typography, and motion.
Backend	Node.js + TypeScript	Use a clearly structured server-side architecture. A dedicated framework such as NestJS is acceptable; avoid duplicating business logic across routes.
Database	PostgreSQL recommended	Store organizations, memberships, templates, document metadata, workflow state, recipients, signature evidence, and audit eve n t s.
Caching	Redis	Use for safe, short-lived cache entries, rate limiting, queue coordination, and transient workflow data where appropriate. Do not use Redis as the source of truth for documents or permissions.
File storage	Local filesystem storage on the PC/server (no S3)	Store logos, letterheads, generated PDFs, attachments, and signed artifacts. Enforce access control and retention policies.
Background work	Queue backed by Redis or a suitable job system	Handle PDF rendering, email delivery, reminders, and other retryable asynchronous tasks
Data interchange	JSON over versioned APIs	Validate request/response schemas; use consistent error formats and API contracts.
Deployment	Docker and environment-based configuration	Provide local development and production configurations, health checks, logs, and repeatable deployment steps.
Testing	TypeScript-compatible unit, integration, and end-to-end testing tools	Automate tenant isolation, permissions, document rendering, and signing workflow tests.

The developer may recommend specific libraries for PDF rendering, schema validation, authentication, email, queues, and signing integration, but must document the choice and its trade-offs before introducing major dependencies.

8. High-Level Data Model

Designanormalized,migration-managedschema. The exact schema is the developer's responsibility, but it must cover at least:

• Organizations and organization branding/settings.

• Platform administrators and organization memberships/roles.

• Registration requests and organization registration policy, if enabled.

• Templates, clauses, uploaded assets, and asset versions.

• Documents, document versions/snapshots, document items/clauses, and generated files.

• Recipients/signers, signing order, signing sessions, consent and evidence records.

• Document links/tokens, expiry/revocation state, and recipient verification state.

• Approval records, workflow events, notifications, and audit events.

• Subscription/usage records if monetization is included in the release.

Every organization-owned entity must be tenant-scoped. Enforce tenant access in server-side services and, where feasible, reinforce it with database-level controls such as row-level security. Never trust a tenant ID supplied by the browser without validating theauthenticatedprincipal'smembershipandpermissions.

9. Security, Privacy, and Reliability

•Implementbasicsecureauthentication,passwordhashingorareputable identity provider, session management, logout, andpasswordreset.Do notaddMFA in theinitialversion.

• Enforce server-side authorization for every organization-scoped read/write action.

• Use least-privilege role permissions and explicit checks for document issue, approval, and signing actions.

• Protect document links with high-entropy tokens; store token hashes where practical; support expiry, revocation, rate limits, and abuse detection.

• Use Use TLS in transit. Store uploaded files on the local filesystem of the PC/server running the application; do not use Amazon S3 or any S3-compatible object-storage service. Restrict access to stored files and protect them with appropriate filesystem permissions and backups.

• Validate file type and content, enforce upload limits, scan files where available, and prevent executable or malicious uploads.

• Keep secrets in environment/secret management, never in source control or client-side bundles.

• Use CSRF protection where applicable, secure cookies, input validation, output encoding, safe headers, and dependency scanning.

• Maintain an append-oriented audit trail for security-sensitive and signing events. Restrict audit access and avoid logging passwords, raw signing tokens, or unnecessary sensitive data.

• Define backup, restore, retention, deletion, incident-response, and operational monitoring procedures.

• Use Redis only for appropriate ephemeral/cache use; apply TTLs and avoid caching sensitive cross-tenant responses without tenant-aware cache keys.

• Do not claim the system can prevent screenshots or photographs of documents; communicate any access controls accurately.

10. API and Engineering Standards

• Usedocumented, versioned APIswith JSONrequest/responsebodies.

• Validate all input at the API boundary with shared schemas where practical.

• Use consistent error codes and human-readable messages without leaking internal details.

• Use database migrations and seed scripts for development/test environments.

• Use environment variables for environment-specific configuration and provide a complete .env.example without real secrets.
• Keep modules small, focused, testable, and maintainable; avoid monolithic files and duplicated logic.
• Use linting, formatting, strict TypeScript, and clear naming conventions.
• Document local setup, environment configuration, migrations, seed data, test commands, deployment, and troubleshooting.
• Use dummy/test data for demos and automated tests; do not put real customer information into seed files.
• Do not commit, push, merge, or run destructive Git operations without explicit authorization from the project owner.

11. Suggested Application Modules

Module	Responsibilities
Public marketing and onboarding	Product information, organization registration, sign-in, onboarding.
Tenant and branding	Organization profile, logo, letterhead, theme, branded auth screens, registration policy.
Members and permissions	Admin/editor membership, invitations, approval of optional registrations, authorization policies.
Template management	Quotation templates, MOU templates, reusable clauses, template versions.
Quotation service	Line items, pricing/tax calculations, numbering, validity, revision history.
MOU service	Structured clauses, parties, dates, signatories, annexures, revisions.
Document rendering	Preview, PDF generation, letterhead placement, snapshot/hash.
Sharing and signing	Secure links, recipient experience, consent, signature capture/provider integration, completion.
Workflow and notifications	Approvals, status transitions, email, reminders, retries.
Audit and reporting	Timeline, security events, document history, usage metrics.
Platform operations	Organization administration, health/usage monitoring, support and abuse controls.

12. Acceptance Criteria

The MVP is not complete until all of the following are demonstrably working:
• A company can register, create its organization, and access its workspace as the initial Organization Admin.
• The admin can upload, preview, replace, and configure branding assets and a letterhead.
• An organization can create a quotation and an MOU, preview them, and generate correctly formatted PDFs.
• The generated documents display the correct organization branding and do not overlap letterhead elements.
• A document can be shared through a secure link with an external recipient who does not need an account.
• The recipient can review the correct version and complete the configured signing process.
• The organization can retrieve the final signed artifact and review the workflow/audit timeline.
• Link expiry and revocation are enforced server-side.
• An editor cannot perform organization-admin actions unless explicitly permitted.
• Optional registration can be disabled, invitation/approval-based, or open according to the configured policy.
• Two test organizations cannot access each other's users, assets, templates, documents, links, or audit records.
• Document versions are preserved and completed/signed artifacts cannot be silently overwritten.
• The UI works on mobile, tablet, and desktop and includes loading, empty, error, and success states.
• Automated tests cover critical authorization, tenant isolation, document state transitions, PDF output, and signing paths.

• The developer delivers setup instructions, architecture notes, API documentation, database migrations, test report, and known limitations.

13. Recommended Delivery Phases

Phase	Scope	Exit condition
1. Foundation	Architecture, database, authentication, organizations, tenancy, admin/editor permissions.	Two organizations can operate independently with tested isolation.
2. White-label setup	Branding assets, letterhead handling, organization settings, branded login and optional registration.	Branding is applied consistently and registration policies are enforced.
3. Document creation	Quotation/MOU templates, forms, preview, PDF generation, versions.	Both document types render accurately and are reproducible.
4. Sharing and signing	Secure links, recipient view, signing, final artifacts, audit history.	End-to-end signing workflow passes functional and security tests.
5. Production readiness	Notifications, reminders, monitoring, backups, accessibility, performance and security tests.	Deployment runbook and test report are accepted.

14. Items to Confirm Before Production

• Confirm the allowed signature-image formats, maximum file size, and whether signature placement should be selectable by the recipient.

• Will subscriptions, billing, usage limits, and free trials be part of the first release or a later phase?

• Which file formats and maximum upload sizes are supported for letterheads and attachments?

• Will custom domains be included in the MVP or deferred?

• What are the retention and deletion policies for completed documents and audit evidence?

• Which languages, currencies, date formats, and tax fields must be supported at launch?

• Will internal approval be mandatory for certain document types or configurable per organization?

15. Developer Handoff Instructions

• Treat this report as the baseline requirements specification. Raise ambiguities before making irreversible architecture decisions.

• Preserve the organization-first white-label model and the distinction between workspace members and external recipients.

• Prioritize end-to-end working functionality, security, correctness, maintainability, and usability over decorative effects.

• Do not use mock interactions as substitutes for core backend behavior. Dummy data is acceptable for demos and tests, but critical workflows must be fully connected to the backend and database.

• Provide a working local development setup and a concise report describing architecture, implemented features, test results, deployment requirements, and unfinished items.

END OF REQUIREMENTS REPORT